

A Color-Based Real-Time Target Detection using OpenCV for DJI Tello Drone

A'dilah Baharuddin
Faculty of Electrical Engineering
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
adilahbaharuddin@gmail.com

Mohd Ariffanan Mohd Basri
Faculty of Electrical Engineering
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
ariffanan@fke.utm.my

Abstract— The utilization of drones has spread into a variety of fields due to the continuous development progress of their technologies. Their applications across military aviation fields into civilian life, where they are performing various tasks such as disaster management, search and rescue operations, and surveillance. Their applications in those operations are possible with the ability to detect the target of the operations. Hence, object detection gained attention in the research community of drones. One of the methods of object detection is color-based detection. The application of the method is not only limited to drones' technology. However, most of the applied color-based methods are designed with a focus on recognizing and labeling the name of the detected colors, but not to detect and recognize the custom target for tracking purposes in the operations. Thus, a color-based detection method without utilizing any advanced machine learning algorithm is proposed to detect the desired target of a drone's operation. A real-time color-based target detection is developed using the OpenCV library. The proposed method is simulated with a webcam and tested with a DJI Tello camera. The results show the target is successfully detected when tested in the same environment as trained. However, the method is sensitive to light intensity and unable to differentiate the target and other objects with the same color. Hence, several improvements to be made are suggested. In the future, for the applications in the drone's target-following operations, the improved algorithm of the proposed method will be put to comparison with a detection using YOLO.

Keywords— Object detection, Color-based detection, OpenCV, UAVs, Drones.

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs), which are commonly referred to as drones, are aircraft that operate autonomously or remotely, without the presence of a pilot on board. They have been developed and utilized since the World War 1 period, when they were initially constructed for armed forces and aviation fields [1,2]. The term "drone, which literally means male bee, is a device developed by U.S Navy commander to be alike to the original device, DH 82B British Queen Bee a device used by the British Royal Navy for target practice [3]. Over the decades, the continuous development of drones has advanced greatly and significantly increased the utilization of drones. The development of drones has made substantial advancements over time, and today drones are classified into a few types, including single-rotor, multi-rotor, fixed-wing, and fixed-wing hybrid VTOL [4,5,6].

In addition to their appeals, such as lower-altitude flight, simple mechanism, and cost efficiency, the advancement of their technological capabilities has led to their application not only for military but also civilian purposes [4,7,8]. Besides, the drones have become significant in facilitating humanity in various ways, as drones are used to perform

various tasks, including monitoring [8], disaster management [9], search and rescue [10], delivering packages [11], and surveillance [12]. These types of tasks may require the drones to possess the ability to detect the target of the scope. For instance, drones are used in the automobile sector in one of the car manufacturers in Germany to detect and locate the set of cars that are ready to be dispatched [2]. This requirement is necessary in surveillance and inspection operations. Hence, drones' object or target detection has become one of the popular scopes in drone research areas.

Object or target detection is one of the common issues in computer vision, and due to its continuous applications, it has become one of the most explored tasks by researchers [13,14]. The methods of object detection are divided into a few methods, and one of the methods is color-based detection. Color is a part of the important characteristics of an image, in which an image is described as consisting of red (R), green (G), and blue (B) color channels [15, 16]. Color-based detection is commonly used to detect and identify the color of target objects in the image or video frame, not only in drones' applications but also in mobile networks [15, 17]. However, color-based detection methods applied to drones are mostly to detect the color of and display its name, instead of real-time detecting and recognizing a custom desired target to be tracked using a purely color-based detection.

Thus, based on image processing workflow, this paper aims to design a real-time, color-based desired target detection on a DJI Tello drone using OpenCV Python's library without implementing any intelligence methods. Based on the fundamental steps in image processing, the proposed method includes image acquisition, preprocessing, segmentation, feature extraction, and recognition. The proposed method is then tested with both the webcam and the drone's camera. The test was conducted with a drone in a non-flying state and positioned on a flat surface.

This paper is constructed into five sections, where section 2 elaborates on target detection steps according to image processing fundamental steps and section 3 elaborates on proposed methods using OpenCV Python library. Section 4 discusses and analyzes the results of the simulation using a webcam and experimentally using DJI Tello's camera, while section 5 concludes and summarizes the paper.

II. TARGET DETECTION

Object or target detection varies into several methods and is generally classified into two main categories, which are traditional and deep learning. Fig. 1 shows the classification of methods applicable to the drone for object or target detection [14]. The traditional methods identify the target by extracting features, which are the small regions of information that are determined based on the appearance or the motion of the target [18]. In contrast, the deep-learning

methods are mostly based on artificial neural network algorithms, which work as if the human brain facilitates decision-making in choosing and recognizing the target [19].

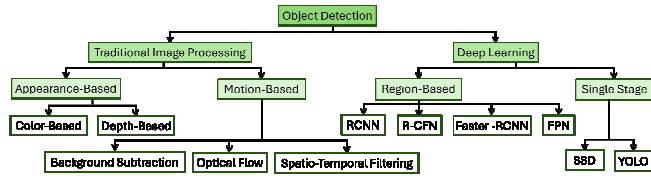


Fig. 1 Object detection methods.

Based on Fig. 1, traditional methods are perceived as image processing, a process of extracting features or useful information from an image [20]. Object or target detection falls in digital image processing, where digital computers are utilized to manipulate the images [20]. The manipulation of the images is based on fundamentals of digital images as shown in Fig. 2, but the application of the entire steps is not necessary [20, 21]. These fundamental steps focus on enhancing and processing the data of images to ease human evaluation or to be used in communication, caching, and machine perception [21].

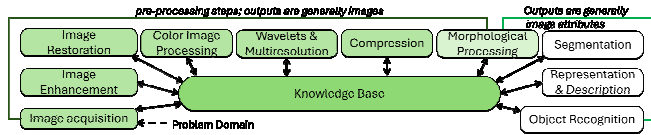


Fig. 2 Fundamental steps of digital image processing.

The fundamental steps chosen in designing color-based detection based on Fig. 2 include image acquisition, enhancement of image, color image processing, morphological processing, segmentation, and object recognition to form the workflow of the proposed method as shown in Fig. 3. The workflow begins with image acquisition, where completely unprocessed images of the target are acquired through the devices' cameras for feature extractions.

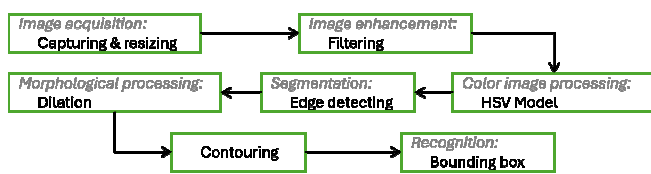


Fig. 3 Workflow of proposed methods based on image processing.

The acquired images can emphasize the hidden details or specific characteristics of the target through the enhancement process [21]. As the images are enhanced, the feature of interest, which is the color of the target, can be extracted with color modeling and processing. Further extraction of color regions is employed with segmentation, where it identifies the boundaries between different regions of color [21].

Morphological processing is an operation where the structuring element is applied to the input image to produce an output image of the same size. The process enhanced other features of the target, such as the size and the shape, and eased the contouring operations, in which the shape or the form of the target is bounded. The feature plays crucial

roles in the last fundamental step, which is recognizing the target process.

III. COLOR-BASED TARGET DETECTION

This paper aims to design a color-based target detection using OpenCV library, a computer vision and machine learning library, on Python-based platform. OpenCV library has over 2500 optimized algorithms and real-world photographs and video comprehension making it a feature to extract information from images in real-time, which allow images to be imported and manipulated in Python [16, 21, 22, 23].

A. Image Acquisition, Resizing, and Filtering

As the proposed method is to be tested with both a webcam and DJI Tello's camera. Thus, the target is captured with both devices, as the features of the target may appear different due to each specification and quality. Fig. 4 shows the captured images by both devices, where both images appear in different tones of color, even though the acquisitions are carried out in the same space. To avoid the distraction in computation, the acquired images are resized to 720x480 resolution, as the size also contributes to the output quality.



Fig. 4 Desired target to be detected

These images are enhanced with filtering methods, to remove contaminated noises in the captured images or features extracted from the images, as noises can affect the detection [9]. It is also a pre-process of edge detection, where the filter applied will blur and smooth the image. A Gaussian blurring is applied using `cv2.GaussianBlur()` to smooth out the image while preserving more edges. To properly choose the kernel size for Gaussian blurring, a trackbar with a scale ranging from 0 to 21 is created, where the smallest size of kernel to be applied is 1x1, and the biggest size will be 43x43. However, the suitable kernel size is expected to not even be 43x43, as an oversized kernel can cause the image to become unrecognizable. The bigger the size of the kernel, the blurrier the image will become. As the kernel size needs to be odd integers, the trackbar is created based on the expression (1).

$$\text{Kernel Size} = (2 * \text{scale's value}) + 1 \quad (1)$$

B. HSV Model

Color models make color specification in a standard way easier. It provides a mathematical representation of image colors that is used to determine the numerical color for mathematical and logical tasks [23]. Among the types of color spaces, the RGB and HSV models are quite commonly used models [9, 18].

The RGB model is the most basic model with a high correlation of red (R), green (G), and blue (B) but is slightly affected by illumination, while the HSV model is very close to human eye perception as it can express brightness, color, and vividness intuitively, which makes it more suitable for color-based image processing [17, 24]. HSV is the acronym for Hue, which defines color; Saturation, which measures the degree of dilution of a pure color by white light; and Value, which indicates the brightness or the intensity of the light-dark color [24, 25].

The input images are captured and stored in the RGB model, where the pixel values are denoted by $B(x, y)$, $G(x, y)$, and $R(x, y)$. The model is converted into HSV image pixel values $H(x, y)$, $S(x, y)$, and $V(x, y)$. Using the OpenCV library, the conversion is carried out with the utilization of the functions `cv2.cvtColor()` and `cv2.COLOR_BGR2HSV`. The conversion produced a binary image that is represented by values from 0 to 255 [26]. Expression (2) represents the threshold values for each HSV channel, where Hue, Sat, and Val represent the values of hue, saturation, and brightness, respectively [15, 19].

$$\begin{cases} 0 \leq \text{Hue} \leq 255 \\ 0 \leq \text{Sat} \leq 255 \\ 0 \leq \text{Val} \leq 255 \end{cases} \quad (2)$$

The binarization process is applied according to expression (3) to create a mask on the image to ensure that only the color of the target is detected [15, 26]. The binary image contains the values of 0 which represent black region and 1 that represents the white region of the pixels.

$$\text{Mask}(x, y) = \begin{cases} 1, \text{ if } 0 \leq \text{Hue, Sat, Val} \leq 255 \\ 0 \end{cases} \quad (3)$$

To obtain suitable HSV values, dynamic trackbars for minimum and maximum values of hue, saturation and value, respectively are created using `cv2.createTrackbar()` and the scales are adjusted until the detection successfully detects only the color of the target. The final values on the adjusted trackbars are chosen as suitable values and fed to the function `cv2.inRange()` to create the binary mask on the image to select the pixels with the values within the ranges chosen from the adjusted trackbars for further processing [16].

C. Edge Detection and Dilation

As the image is smoothed by filtering, the process of edge detecting can be performed. Edge detection is an image segmentation, where it divides the image into sections with own properties' set each to identify the significant brightness changing point on abrupt or discontinuous gray scale [17, 21]. Hence, the image is first converted into grayscale using the function `cv2.cvtColor()` and `cv2.COLOR_BGR2GRAY`.

The Canny edge detection, a commonly used operator that is equipped with filtering, enhancing and detecting functions, is applied with function `cv2.Canny` [17]. The thresholds for edge detection are determined through dynamic threshold adjustment in real-time using a trackbar with a range of 0 to 255 is applied.

The morphological process is then conducted upon the noise removal and edge detection processes completed. For this work, the dilation operator, a process of enlarging the foreground structures of an image is applied using `cv2.dilate` [24]. As the application of dilation required to use kernel size, the same expression in (1) is reused again to build a trackbar to determine the suitable kernel size for the dilation process.

D. Image Contouring and Bounding Box

As the Canny operation is applied in the edge detection process, a better accuracy of image contouring is expected, as the localization of the object in the image becomes much easier. The contour operator is applied using `cv2.findContours()`, along with retrieval method that retrieves only the parents' contours, `cv2.RETR_EXTERNAL`, and approximation method that store all contour points, `cv2.CHAIN_APPROX_NONE`. To overlay the contour on the original image in RGB model, function `cv2.drawContours()` is applied.

As the target is successfully identified, a bounding box is created using function `cv2.boundingRect()` to indicate the target detected by the algorithm. The information used to create the bounding box is obtained from the contouring process, where the function `cv2.arcLength()` is utilized to find the perimeter of the object and function `cv2.approxPolyDP()` is used to approximate the contour shape to another shape.

E. Evaluation Metric

The designed detector is evaluated by measuring the processing time, which reflects the duration required for each frame to identify the existence of the target within it. The success of the designated algorithm in analyzing and processing input data is evaluated by evaluating the frame rate whenever the target is detected in the captured frame. Expression (4) denotes the calculation of frame rate (fps), where the number of frames to be captured is divided by the duration required for frame processing.

$$\text{frame rate} = \text{frame number} / \text{duration} \quad (4)$$

A higher frame rate can generally be desirable as it improves visual experience by providing a more precise portrayal of motion, which indicates the system can process detections in real time without latency.

IV. RESULTS AND DISCUSSION

The proposed method is simulated with webcam detection. The values of HSV, kernel sizes and thresholds are determined with trackbars adjustment during the training using the image stored from acquiring and resizing process. The values are then used as the preset values during the real-time simulation.

The method is then put to experimental with non-flying DJI Tello's camera. The experiment is conducted with values obtained from both simulation data and experimental data. The trackbars are adjusted during training to obtain suitable value and further adjustments are made upon requirement during the real-time application. Fig. 5 shows

the trackbars built, where “Thresh 1” and “Thresh 2” trackbars represent the thresholding for edge detection, while “BlurKernel” and “DilKernel” represent the kernel size trackbars for Gaussian blurring and dilation, respectively.



Fig. 5 Trackbars for values adjustment during training and experiment.

As the target used in this paper is made of two different colors, the detection can only be used to detect one color in order to ensure the detection is precise. Thus, the yellow color is chosen as the color to be identified from the target.

The evaluations of detection are printed on the final output image when the target appears in the frame. The frame to be shot in the processing time is set to 1 frame, which represents a still image and 120 frames, which represents a slow-motion video.

A. Simulation Results

The proposed method is simulated using web camera. Fig. 6 shows the output of each operation in the image processing, where these outputs are achieved through the adjustment on the trackbars. The output image shows the contour on the edge of the target and the bounding box around the target as desired.

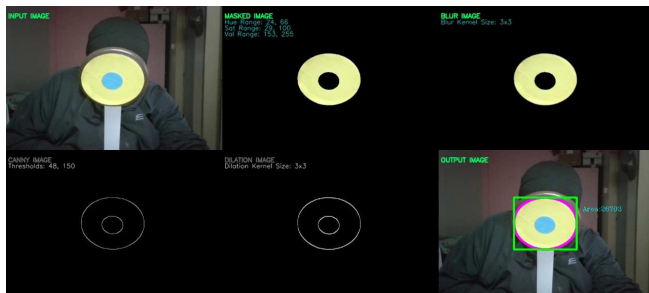


Fig. 6 Outputs of each operation in image processing.

The final values of trackbars adjustment that yield the outputs as in Fig. 6 are listed in Table 1. The values of HSV, thresholds for edge detection and kernel size for blurring and dilation that obtained from the training using web camera captured image are used as preset values.

TABLE 1. The values obtained from training for a web camera.

Parameters	Values
Hue (Minimum, Maximum)	(24, 66)
Saturation (Minimum, Maximum)	(29, 100)
Value (Minimum, Maximum)	(153, 255)
Kernel Size [Blurring, Dilation]	[3x3, 3x3]
Edge Detection (Threshold1, Threshold2)	(48, 150)

The values in Table 1 are then utilized in real-time simulation of target detection. The simulation is conducted in the same indoor environment. Fig. 7 shows the output of real-time detection simulated with a web camera, even though the target is not at the same angle as in training

image. The output of the simulation shows the target successfully detected as the edge is contoured and the target is bounded with the box.

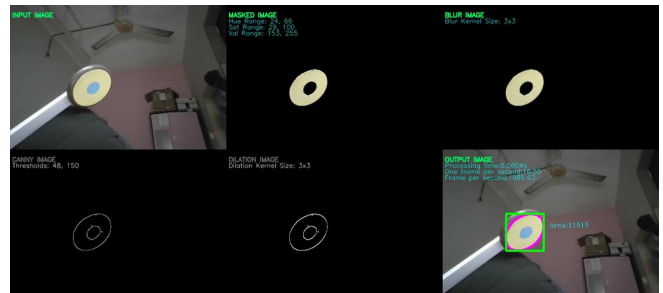


Fig. 7 The outputs of real-time detection with a web camera.

The detection of the target is processed at 0.0604s, with 16.55fps for a frame and 1985.63fps for 120 frames. These show the efficiency of detection as high fps are recorded over small processing time. Several simulations are conducted with the target are moved around in entire frame and tilted to different angles. The simulation recorded the processing time varied within the range of 0.01s to 0.07s, with fps in one frame varied in the range of 30fps to 60fps and 3000fps to 8000fps for 120 frames.

However, when the target is exposed to different intensity of light, the algorithms of the detection are not performing as well as in the same light exposure on the target during training. Fig. 8 shows the results of detection in when the target is exposed to different light intensity and changed the tone of the target. The changes have caused the whole operation to be confused and unsuccessful in processing to precisely detect the desired target as one.



Fig. 8 The results of detection in different light intensity.

Even so, the processing of time recorded by the detection is 0.0219s with fps of 45.6 for one frame captured and 5471.49 for 120 frames captured, which indicates the detection is performing in high efficiency.

B. Experimental Results

The proposed method is then tested with DJI Tello's camera. The values of parameters to be used for drone's operation are modified accordingly with the trackbars. Fig. 9 shows the output of training sessions with image capture by Tello's camera with satisfying results, as the target is detected and identified by the operations.

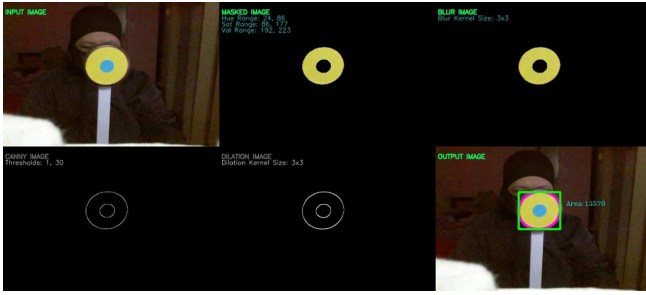


Fig. 9 The outputs of training session with DJI Tello.

Table 2 listed the values of required parameters used in the training sessions, where these are then utilized as the preset values for real-time experiments with DJI Tello's camera.

TABLE 2. The values obtained from training for DJI Tello.

Parameters	Values
Hue (Minimum, Maximum)	(23, 34)
Saturation (Minimum, Maximum)	(124, 242)
Value (Minimum, Maximum)	(116, 168)
Kernel Size [Blurring, Dilation]	[3x3, 3x3]
Edge Detection (Threshold1, Threshold2)	(1, 32)

Fig. 10 shows the output of experimentation with DJI Tello's camera and the experiment is conducted indoors in the same environment as the training session. Each operation is successfully processing the image. The output image shows that the edge of the target is contoured as it should. The box also successfully bounded on to the target.

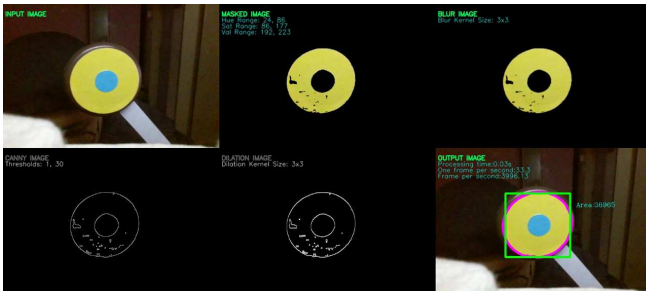


Fig. 10 The output of real-time experiment with DJI Tello's camera.

The processing time for the experiment conducted in Fig. 10 reflects a good efficiency of detection where the time taken for the detection to be processed is only 0.03s, with fps at 33.3 for one frame captured and 3996.13 for 120 frames captured. Several sessions of experiment are conducted also reflect the result of good detection where the processing time varies in the range of 0.01s to 0.04s, with fps of one frame captured varied from 25fps to 80fps and the fps of 120 frames captured varied from 3100fps to 9600fps.

The experiment is next conducted in slightly brighter surrounding, where the target is exposed with brighter light than training session. However, the same situation as in simulation occurred where the difference in light intensity and exposure affects the performance of the detection occurs during the experiment, as shown in Fig. 11, where only partial of the target is detected.

The detection imprecisely detected the target into several objects instead of one object. Despite the confusion in detecting the target as one, the processing time of 0.028s,

and the fps values are recorded with 35.71fps for one frame captured and 4285.59fps for 120 frames captured.



Fig. 11 The output of real-time experiments in low light intensity.

C. Discussion

The training sessions for simulation and experiments must be conducted separately, as both devices captured images with different resolutions. Thus, different values in color space are expected to be used by the devices.

In general, the proposed method of object detection successfully detects the target as it is trained to. The algorithm is trained to detect the target using the yellow color of the target. Both simulation and experiment show that if the applications are applied in the same surroundings as the training session, the detection of the target works nicely. The short processing time and high frame rate recorded by both simulation and experiment sessions proved the reliability of the algorithm in recognizing the target.

However, when a change occurred in the light intensity, the algorithms face confusion where the target fails to be detected as a single object. This is due to the high dependency of the color model on the light intensity. A slight change applied to the intensity may result in the changes in the HSV values on certain of the target. Thus, the data applied to the algorithm is no longer accurate, as the color of the target appeared in the frame might also have some certain part with the values of HSV that is out of the range obtained from the training. Therefore, a technique such as lighting or illumination normalization to limit the light source, or other color correction techniques to the devices should be integrated into the model in order for it to adjust the captured resolution to match the one used in the training sessions.

In the meantime, the proposed method relies hardly only on the color for detection that it causes the algorithm to detect any object with the same color as the target. The application of edge detection successfully detects the edge shape of the target. However, as there is no classification of target in the operations, the algorithm will detect the edge of any object with the same color as the target. Hence, another feature such as the specific shape or texture should be added in classifying the target. Therefore, the algorithm will be more robust as the target becomes much more different than other objects with the same color.

The proposed method is applicable in detecting the color, but as it has no ability to differentiate the target from other objects that possessed the same color as the target, it is not effective enough to be applied for target-only identification and target-following operations. Therefore,

the whole proposed method needs to be improved by integrating several other techniques for more accurate and reliable detection.

V. CONCLUSION

This work aims to design a target detection purely with a color-based detection method. OpenCV is utilized as its applications are inexpensive, easy, and fast processing. The operations applied according to the fundamentals of image processing have successfully performed their respective tasks to produce the target detection. Based on the processing time and frame rates, the target is successfully detected as per trained, where short processing time is recorded along with high fps values in every detection. However, as the method is highly sensitive toward light intensity and easily confuses other objects with the same color as the target, the method is not reliable for target-following operations. Thus, several improvements and additional techniques should be applied to the proposed method to make it more robust and more reliable. In the future, the utilization of OpenCV in object detection of a drone will be designed with a YOLO method, and the performance of the method will be put into comparison with the improved algorithm of the proposed method for further improvement.

ACKNOWLEDGMENT

The authors would like to thank the Ministry of Higher Education (MOHE) through Fundamental Research Grant Scheme (FRGS/1/2021/TK0/UTM/02/56) and Universiti Teknologi Malaysia through UTMFR (Q.J130000.3823.22H67) for supporting this research.

REFERENCES

- [1] J. Kandpal, P. K. Mishra, B. Chilwal, A. K. Singh, and G. Balutia, "Introduction to Drone Technologies," in *Revolutionary Applications of Intelligent Drones*, M. Angurala and V. Khullar Eds. New York: Nova Science Publishers, Incorporated, 2022, ch. 1 sec. 1, pp. 1 - 23.
- [2] P. K. Mishra, J. Kandpal, A. K. Singh, G. Balutia, B. Chilwal, and B. Prasuna, "Drones for Society and Industry," in *Revolutionary Applications of Intelligent Drones*, M. Angurala and V. Khullar Eds. New York: Nova Science Publishers, Incorporated, 2022, ch. 2, pp. 25-44.
- [3] Y. Akbari, N. Almaadeed, S. Al-maadeed, and O. Elharrouss, "Applications, databases and open computer vision research from drone videos and images: a survey," *Artificial Intelligence Review*, vol. 54, no. 5, pp. 3887-3938, 2021/06/01 2021, doi: 10.1007/s10462-020-09943-1.
- [4] A. Agarwal, C. Mohanta, and S. N. Mehta, "Drone Technologies," in *Drone Technology*, 2023, sec. State-of-the-Art, Challenges, and Future Scope, pp. 1-19.
- [5] T. Ritika, J. Sheilza, and B. Sakshi, "Quadrotors In The Present Era: A Review," *Information Technology in Industry*, vol. 9, no. 1, pp. 164 - 178, 2021, doi: <https://doi.org/10.17762/itii.v9i1.116>.
- [6] T. Koç Mehmet, "Drone Technologies and Applications," in *Drones - Various Applications*, C. Dragan Ed. Rijeka: IntechOpen, 2023, p. Ch. 1.
- [7] H. Taneja and A. Sharma, "Drones and Their Utilities Using Emerging Technology," in *Revolutionary Application of Intelligent Drones*, M. Angurala and V. Khullar Eds.: Nova Science Publishers, Inc, 2022, ch. 4, pp. 55 - 66.
- [8] A. M. Yunus, A. H. Hamzah, and F. A. Md. Azmi, "Drone Technology as A Modern Tool in Monitoring the Rural-Urban Development," *IOP Conference Series: Earth and Environmental Science*, vol. 540, no. 1, p. 012076, 2020/07/01 2020, doi: 10.1088/1755-1315/540/1/012076.
- [9] X. Xie, J. Wang, Y. Zhang, J. Xin, L. Mu, and X. Xue, "A UAV Forest Fire Detection Method Based on Dual-Light Vision Images," presented at the 2023 CAA Symposium on Fault Detection, Supervision and Safety for Technical Processes (SAFEPROCESS), Yibin, China, 2023.
- [10] R. Ashour, S. Aldhaheri, and Y. Abu-Kheil, "Applications of UAVs in Search and Rescue," in *Unmanned Aerial Vehicles Applications: Challenges and Trends*, M. Abdelkader and A. Koubaa Eds. 169 - 200: Springer Nature Switzerland, 2023
- [11] L. Di Puglia Pugliese, F. Guerriero, and G. Macrina, "Using drones for parcels delivery process," *Procedia Manufacturing*, vol. 42, pp. 488-497, 01/01 2020, doi: 10.1016/j.promfg.2020.02.043.
- [12] N. Dilshad, J. Hwang, J. Song, and N. Sung, "Applications and Challenges in Video Surveillance via Drone: A Brief Survey," presented at the 2020 International Conference on Information and Communication Technology Convergence (ICTC), Jeju, South Korea, 2020, 2020.
- [13] S. Abdallah, "Converting a DJI Tello Quadcopter into a Face-follower Machine Using the Haar Cascade with PID Controller," *Cihan University-Erbil Scientific Journal (cuesj)*, vol. 7, no. 2, pp. 54 - 59, Dec 2023 2023, doi: 10.24086/cuesj.v7n2y2023.pp54-59.
- [14] A. Ramachandran and A. K. Sangaiah, "Areview on object detection in unmanned aerial vehicle surveillance," *International Journal of Cognitive Computing in Engineering*, vol. 2, pp. 215 - 228, 2021, doi: 10.1016/j.ijcce.2021.11.005.
- [15] B. Marton, "History, types, application and control of drones," *Safety and Security Science Review*, vol. 4, no. 3, pp. 1-13, Sep 21 2022, doi: <https://biztonsagtudomanyi.szemle.uni-obuda.hu/index.php/home/article/view/222/202>.
- [16] V. Yevsieiev, A. Abu-Jassar, and S. Maksymova, "Object Recognition and Tracking Method in the Mobile Robot's Workspace in Real Time," *Technical science research in Uzbekistan*, no. 2, pp. 115 - 124, 2024. [Online]. Available: <https://openarchive.nure.ua/handle/document/27494>.
- [17] D. T. Joy, G. Kaur, A. Chugh, and S. Baskar Bajaj, "Computer Vision for Color Detection," *International Journal of Innovative Research in Computer Science & Technology (IJIRCST)*, vol. 9, no. 3, pp. 53 - 59, may 2021 2021, doi: 10.21276/ijircst.2021.9.3.9.
- [18] G. Liu, C. Zhang, Q. Guo, and F. Wan, "Automatic Color Recognition Technology of UAV Based on Machine Vision," presented at the 2019 International Conference on Sensing, Diagnostics, Prognostics, and Control (SDPC), Beijing, China, 2019.
- [19] W. Li, X. S. Feng, K. Zha, S. Li, and H. S. Zhu, "Summary of Target Detection Algorithms," *Journal of Physics: Conference Series*, vol. 1757, no. 1, p. 012003, 2021/01/01 2021, doi: 10.1088/1742-6596/1757/1/012003.
- [20] N. O. Mahony *et al.*, "Deep Learning vs. Traditional Computer Vision," *ArXiv*, vol. abs/1910.13796, 2019.
- [21] M. Lather and P. Singh, "Image Processing: What, How and Future," in *Computational Methods and Data Engineering*, Singapore, V. Singh, V. K. Asari, S. Kumar, and R. B. Patel, Eds., 2021 vol. 1227: Springer Singapore, in Advances in Intelligent Systems and Computing, pp. 305-317, doi: 10.1007/978-981-15-6876-3_23. [Online]. Available: https://doi.org/10.1007/978-981-15-6876-3_23
- [22] S. Mishra, V. Verma, N. Akhtar, S. Chaturvedi, and Y. Perwej, "An Intelligent Motion Detection Using OpenCV," *International Journal of Scientific Research in Science, Engineering and Technology (IJSRSET)*, vol. 9, no. 2, pp. 51 - 63, Mar - April 2022 2022, doi: 10.32628/IJSRSET22925.
- [23] R. Arali, Sumitha M, Swathi P, V. Halawal, and A. Naik, "Colour Detection from Image," *International Journal of Advances in Engineering and Management (IJAEM)*, vol. 3, no. 7, pp. 2164 - 2171, 7 July 2021 2021, doi: 10.35629/5252-030721642171.
- [24] U. P. Nagane and A. O. M. Dr, "Moving Object Detection and Tracking Using Matlab," *Journal of Science & Technology (JST)*, vol. 6, no. Special Issue 1, pp. 63-66, 08/16 2021, doi: 10.46243/jst.2021.v6.i04.pp63-66.
- [25] M. Solilo, W. Doorsamy, and B. Paul, *UAV Power Line Detection and Tracking using a Color Transformation*. 2021, pp. 1-5.
- [26] K. A. M. Said and A. B. Jambek, "Analysis of Image Processing Using Morphological Erosion and Dilaton," *Journal of Physics: Conference Series*, vol. 2071, no. 1, p. 012033, 2021/10/01 2021, doi: 10.1088/1742-6596/2071/1/012033.